Development of a Fast ABAQUS Post-Processing Plugin for Computing Average Field Quantities of a Microstructure

A report on a bulk-data-block-based, homogenisation-oriented Abaqus/CAE post-processing plug-in developed at Microsolidics

AJAY PRATAP SINGH
B.Tech, 4th Year, Department of Mechanical Engineering
Indian Institute of Technology Guwahati

August 2026

Abstract

Predicting the effective mechanical response of heterogeneous materials from their underlying microstructure is central to multiscale computational mechanics. A common bottleneck in this workflow is not the finite-element solution itself but the post-processing step that follows it — extracting field variables such as stress, strain and energy densities from an Abaqus Output Database (.odb) and volume-averaging them over a Representative Volume Element (RVE) and over its constituent phases. When done element-by-element in the Abaqus/Python kernel, this step scales poorly with mesh size and becomes the practical limiting factor for high-resolution or batch RVE studies.

This report documents the design and development of an Abaqus/CAE plug-in, built around a custom Abaqus GUI Toolkit (AFX) dialog and a generalized kernel-side extraction module, that automates this averaging step for arbitrary field-output variables. The central contribution is a shift from the conventional element-by-element / node-by-node Python loop to Abaqus's bulkDataBlocks interface, which returns field data for an entire output request as contiguous NumPy-compatible arrays. Combined with vectorised NumPy operations, this bulk-data-block strategy removes the per-object Python-C++ call overhead that dominates the runtime of loop-based scripts, giving an order-of-magnitude reduction in post-processing time on realistic microstructural meshes.

The plug-in automatically classifies each requested variable by its storage position (integration point, centroid, whole-element, nodal or surface) and dispatches it to the appropriate generic volume- or area-weighted averaging routine, so that a user only has to type variable names (e.g. S, E, PEEQ, SENER) rather than write new code for every quantity of interest. The report covers the theoretical background of multiscale modelling and RVE-based homogenisation, the tools used for microstructure generation and boundary-condition application, the architecture of the developed GUI and its underlying kernel script, the governing averaging equations, and the motivation and mechanics of the bulk-data-block acceleration strategy.

Table of Contents

Abstract 2
Table of Contents 3
1. Introduction 1
2. Multiscale Modelling and the Microscale RVE 1
2.1 The Representative Volume Element (RVE) 1
2.2 Types of Multiscale Modelling 2
2.3 Which Approach This Plug-in Supports 2
3. Requirement of Multiscale Modelling 3
4. Microstructure Generator Tools 3
5. Boundary Condition Tool 4
6. Post-Processing Tool for Microstructure — The Conventional Approach 4
7. Motivation Towards the Strategy of Using a Bulk-Data-Block, Faster Tool 5
8. The Bulk Data Block Option 6
8.1 What a bulk data block is 6
8.2 Why it is faster 6
8.3 How it is used in the developed extractor 6
9. Field Quantities Computed and the Governing Equations 7
9.1 Volume-averaged stress and strain 7
9.2 Whole-element (already-integrated) quantities 7
9.3 Von Mises equivalent stress and principal values 8
9.4 Tensor storage convention — Mandel–Kelvin ordering 8
9.5 Volume fraction of each phase 8
9.6 Surface-averaged quantities 8
9.7 History and whole-model / modal quantities 8
9.8 Effective (homogenised) stiffness — identified future extension 8
10. Plug-in Architecture and Implementation 9
10.1 Overview 9
10.2 GUI dialog (my_gui.py) 9
10.3 Kernel extraction module (data_extractor.py) 10
10.4 Company branding 10
11. Typical Workflow 10
12. Discussion 11
13. Conclusion and Future Scope 11
13.1 Conclusion 11
13.2 Future Scope 11
References 12

1. Introduction

Components made of composite materials derive their macroscopic (bulk) mechanical behaviour from geometric and material features that exist at a much smaller length scale — the microstructure. Directly resolving this microstructure inside a full-scale component simulation is computationally prohibitive, so engineers instead simulate a small, statistically representative block of the microstructure (the RVE), apply appropriate boundary conditions, and average the resulting fields to obtain equivalent bulk properties. This RVE-based homogenisation workflow is the backbone of multiscale modelling and is widely used in Abaqus/CAE through user-scripted pre- and post-processing.

This project was undertaken to build a reusable, general-purpose Abaqus plug-in for the last stage of this workflow: automated, fast extraction and volume-averaging of field quantities from an RVE simulation's output database. The plug-in consists of a custom AFX-based graphical dialog (my_gui.py, registered through my_gui_plugin.py) that lets a user browse for an .odb file, pre-inspect its contents, and specify a step name, an output file name and a comma-separated list of field variables, and a kernel-side generalized extractor module (data_extractor.py) that performs the actual averaging using Abaqus's bulkDataBlocks API for speed.

2. Multiscale Modelling and the Microscale RVE

Multiscale modelling refers to the class of simulation strategies that link material behaviour across two or more characteristic length scales — typically a microscale (grains, fibres, inclusions) and a macroscale (the engineering component). Rather than meshing every microstructural feature inside the full component, a small, geometrically and statistically representative sub-volume of the microstructure is simulated separately and its computed effective (homogenised) response is handed up to the macroscale model, either as an effective material property (linear elasticity) or through a scale-bridging scheme such as computational homogenisation / FE^2 (nonlinear, history-dependent behaviour).

2.1 The Representative Volume Element (RVE)

The RVE is the fundamental building block of this strategy. Hill [1] originally defined it, for statistically homogeneous media, as a sample that is (a) structurally entirely typical of the whole mixture on average, and (b) large enough to contain a sufficient sampling of microstructural heterogeneities so that the apparent overall properties are independent of the applied boundary conditions and insensitive to which particular realisation of the microstructure is used. Kanit et al. [2] later gave this a quantitative, statistical footing, showing that the required RVE size depends on the property being homogenised, the phase contrast, the volume fraction of the phases and the number of independent realisations averaged.

A microstructural RVE model, as used in this project, therefore consists of:

- A geometric domain Ω containing one or more material phases (matrix, inclusions, fibres, grains …), each assigned its own constitutive material model and identified in Abaqus through a section/material assignment (a “phase” in the terminology used by the plug-in).

- An imposed loading or displacement state on the boundary ∂Ω of the RVE (kinematic uniform, static uniform, or periodic boundary conditions).
- Volume-averaged response quantities (stress, strain, energy) computed after solving, which represent the equivalent homogeneous point response of the macroscale material.

Formally, for any local field quantity f(x) defined over the RVE domain Ω of volume V, the macroscopically-equivalent (homogenised) quantity is its volume average:

$$\langle f \rangle = (1/V) \int_{\Omega} f(x)\, dV \quad (1)$$

which, on a finite-element discretisation, is approximated by a weighted sum over Gauss / integration points or elements — precisely the quantity the developed plug-in computes automatically for any requested field variable.

2.2 Types of Multiscale Modelling

Multiscale strategies are usually grouped into three broad families, based on how information is exchanged between the microscale (RVE) and the macroscale (component) models:

- Sequential (hierarchical / one-way) multiscale modelling: the RVE is solved separately and in advance. Its homogenised response — an effective stiffness, strength, or a calibrated constitutive law — is extracted once (or for a small design-of-experiments set of RVEs) and then handed up to the macroscale model as an ordinary material property. The two scales are solved once each and are not revisited together; this is the classical route for linear-elastic or weakly nonlinear, scale-separated problems, and is the approach followed in periodic-homogenisation and RVE-averaging methods [3, 6, 7].
- Concurrent multiscale modelling (computational homogenisation / FE²): no explicit macroscale material law is written down at all. Instead, at every macroscale integration point, in every load increment of the macroscale analysis, a full microscale RVE boundary-value problem is solved on the fly to supply the stress and consistent tangent needed by the macroscale solver, and the two scales are solved together, iteration by iteration. This captures nonlinear, history- and path-dependent microstructural behaviour exactly, but is computationally very expensive because one macroscale analysis now contains thousands of nested RVE solves [12, 13].
- Semi-concurrent / data-driven (surrogate) multiscale modelling: a hybrid in which many RVEs are solved offline (sequentially, as in the first category) to train a fast surrogate — a reduced-order model, a neural network, or a clustering-based approximation — which is then queried concurrently inside the macroscale analysis in place of a full RVE solve, aiming to combine FE² accuracy with sequential-homogenisation speed.

2.3 Which Approach This Plug-in Supports

The tool developed in this project belongs to the sequential (hierarchical) family. It post-processes one already-solved RVE simulation — opening its .odb file, averaging the field quantities over each phase and the whole RVE (Equations 1, 3 and 4), and writing the homogenised result to a text file, entirely independently of, and after, any macroscale analysis. It is not invoked repeatedly from inside a running macroscale job the way an FE² microscale solver would be; there is no online coupling between scales. This is precisely why the speed of a single post-processing pass (Sections 6–8) matters so much for the

intended use case: in a sequential workflow, the RVE is typically re-run and re-post-processed many times over — for different microstructural realisations, volume fractions, or load cases — to build up the effective-property data that is then handed to the macroscale model, so a slow post-processing step is repeated, and its cost multiplies, exactly as many times as the study requires.

3. Requirement of Multiscale Modelling

A full-field simulation that explicitly resolves every microstructural feature (every fibre, particle or grain) inside a real, component-scale geometry is, for almost all practical materials, computationally intractable: mesh sizes would have to be several orders of magnitude finer than the component, driving degree-of-freedom counts and solve times far beyond available hardware. Multiscale / RVE-based modelling exists specifically to make such analyses feasible, and is required for the following reasons:

- Computational tractability: replacing millions of microstructural features with a homogenised bulk property (or a small RVE evaluated at each macroscopic integration point) reduces problem size by orders of magnitude.
- Design of new composite materials: evaluating how a change in microstructure (volume fraction, particle/fibre morphology, phase properties) changes bulk behaviour requires rapidly re-running the RVE workflow, which is only practical if each evaluation is fast.
- Physically-grounded macroscale constitutive models: homogenised RVE responses can be used to calibrate or directly drive (FE^2) macroscale material models, giving predictions that are traceable to the actual microstructure rather than to a curve-fit.
- Property prediction without physical testing: effective stiffness, strength and thermal properties of a new composite material can be estimated computationally before manufacturing test coupons.
- Sensitivity and optimisation studies: because RVE simulations are comparatively cheap, they can be run in batches (different volume fractions, morphologies, boundary conditions) to build design charts — something only possible if pre-processing, solving and, crucially, post-processing are all fast and largely automated.

It is this last point — the need to run and post-process many RVE simulations quickly — that directly motivates the plug-in described in this report: a slow, manual post-processing step erodes the very speed advantage that multiscale/RVE modelling is supposed to provide over full-field simulation.

4. Microstructure Generator Tools

Before an RVE can be solved, its geometry must be generated: the spatial arrangement, size distribution and volume fraction of each phase (inclusions, fibres, grains) inside the unit cell. Commonly used microstructure-generation approaches, which precede the extraction step this project focuses on, include:

- Random sequential addition (RSA) / random packing algorithms, which place non-overlapping inclusions (spheres, ellipsoids, fibres) at random positions until a target volume fraction is reached.

- Voronoi tessellation, used to generate polycrystalline / cellular microstructures such as those studied by Kanit et al. [2] for statistical RVE-size determination.
- Digital-image-based / voxel reconstruction, where a real microstructure is imaged (X-ray CT, SEM) and converted directly into a finite-element or voxel mesh.
- Parametric CAD generation inside Abaqus/CAE (Part and Sketch modules) driven by Python scripting, used for periodic unit cells with regular fibre or particle arrangements.

Each phase produced by these generators is subsequently assigned a section and a material in Abaqus/CAE; the developed post-processing plug-in later reads exactly these section/material assignments back out of the .odb (see Section 10) to automatically identify how many phases exist and to which material each belongs, without the user having to specify this manually.

5. Boundary Condition Tool

The averaged field quantities computed from an RVE are only meaningful homogenised properties if the RVE has been loaded with boundary conditions consistent with the Hill–Mandel macro-homogeneity condition, which requires the volume average of the local energy to equal the product of the average stress and average strain:

$$\langle \sigma_{ij}\varepsilon_{ij} \rangle = \langle \sigma_{ij} \rangle \langle \varepsilon_{ij} \rangle \quad (2)$$

Three classical boundary-condition types satisfy this condition and are typically implemented as a dedicated “boundary condition tool” (a script that ties nodes / applies equations) alongside the microstructure generator:

- Kinematic Uniform Boundary Conditions (KUBC) — a linear displacement field $u_i = \bar{\varepsilon}_{ij} x_j$ is prescribed on $\partial\Omega$; gives an upper (stiff) bound on the effective stiffness.
- Static Uniform Boundary Conditions (SUBC) — a uniform traction $t_i = \bar{\sigma}_{ij} n_j$ is prescribed on $\partial\Omega$; gives a lower (compliant) bound.
- Periodic Boundary Conditions (PBC) — opposite faces of the RVE are tied through equal-and-opposite displacement (or multi-point constraint / equation) relations that enforce periodicity of the fluctuating displacement field; PBC typically gives the tightest, most accurate estimate of effective properties for a periodic or statistically periodic microstructure and is the most common choice in Abaqus-based RVE work [10, 11].

In Abaqus/CAE, boundary-condition tools of this kind are usually implemented as Python scripts that automatically identify matching node pairs on opposite RVE faces/edges/vertices and generate the corresponding *EQUATION constraints or reference-point-driven multi-point constraints, so that the same tool can be re-applied to any newly generated microstructure without manual node-picking. While the boundary-condition and microstructure-generation tools are upstream of the work reported here, they define the loading history that the post-processing plug-in subsequently averages over each analysis step and frame.

6. Post-Processing Tool for Microstructure — The Conventional Approach

Once an RVE has been meshed, assigned phases, loaded through one of the boundary-condition schemes above, and solved, the results are written to an Abaqus Output Database (.odb) as field outputs (S, E, PEEQ, SENER …) evaluated at every integration point / node / element, for every requested frame of every step. The task of the post-processing tool is to reduce this large, spatially and temporally resolved dataset down to a handful of physically meaningful, per-phase and overall (RVE-wide) averaged quantities — the equivalent homogeneous-material response.

The conventional way of doing this inside the Abaqus/Python (odbAccess) kernel is to iterate directly over the field-output value objects: opening the field output for a given variable and frame, then looping, in pure Python, over every Value object it contains (one per integration point / node), reading its .data, accumulating a running sum and a running weight, and only converting to an average once the loop has finished. A representative fragment of this conventional pattern is:

```
for v in fieldOutput.values:     total += v.data * v.instance... # weight lookup
count += 1 average = total / count
```

This works correctly, but every iteration of the loop crosses the Python–C++ boundary of the Abaqus kernel once per value object, and for even a moderately fine RVE mesh (tens of thousands of elements, several integration points per element, several field variables, many frames) the number of such crossings quickly reaches into the millions. Because this call overhead — not the arithmetic itself — dominates the running time, conventional value-by-value post-processing scripts become the slowest step of the whole simulate-then-analyse workflow, especially when the same script has to be re-run for a batch of RVE realisations of the kind needed for RVE-size or design studies (Section 3).

7. Motivation Towards the Strategy of Using a Bulk-Data-Block, Faster Tool

Given the requirement identified in Section 3 — running and post-processing many RVE simulations quickly enough to make multiscale studies practical — the guidance for this project was specifically to move away from the conventional per-value looping pattern of Section 6 and instead build the extraction tool around Abaqus's bulkDataBlocks interface, and to demonstrate and document the resulting reduction in post-processing time.

The motivation for this can be summarised as follows:

- Post-processing, not solving, was identified as the practical bottleneck for repeated RVE studies: solving is handled efficiently by Abaqus's own compiled solver, but conventional Python post-processing re-introduces an interpreted, object-by-object loop precisely where speed is most needed.
- A generic, variable-agnostic tool was needed so that a new field quantity (e.g. adding HFL or CTF1 to a study) does not require writing a new hand-coded extraction block, but this generality must not come at the cost of speed — which rules out simply generalising the conventional loop.
- The tool needed to remain usable from a simple GUI dialog (Section 10) by a non-scripting user, so that variable selection, pre-processing/inspection and the compute step are all point-and-

click, while the heavy averaging work underneath runs as fast as the Abaqus scripting interface allows.

- Because the same averaging logic has to run once per phase, per component, per frame and per requested variable, even a modest per-call overhead is multiplied many times over a full study; eliminating that overhead was therefore judged to give the largest practical speed-up for the least added complexity.

8. The Bulk Data Block Option

8.1 What a bulk data block is

A FieldOutput object in the Abaqus scripting interface stores its data internally in contiguous, homogeneous blocks — one block per (instance, section-point/position, element-or-node-type) combination — rather than as one Python object per value. The .bulkDataBlocks attribute exposes these blocks directly: each block's .data is returned as a single array-like object containing every component of every entity in that block in one call, along with parallel arrays such as .elementLabels, .nodeLabels or .integrationPoints. Reading a variable through .bulkDataBlocks therefore costs a small, fixed number of Python–kernel calls (one per block) instead of one call per value.

8.2 Why it is faster

The runtime cost of the conventional loop in Section 6 is dominated by two things: (i) the fixed overhead of a Python-level call into the Abaqus kernel for every single Value object, and (ii) the cost of Python-level arithmetic performed one scalar/tuple at a time. The bulk-data-block strategy addresses both:

- Fewer kernel calls: a field output with tens of thousands of integration points is retrieved in as many calls as there are bulk data blocks (typically one to a few dozen, depending on how many instances / element types are present) rather than one call per integration point — often a 3–4 order-of-magnitude reduction in the number of Python–kernel round trips.
- Vectorised arithmetic: once a block's .data is converted to a NumPy array (as np.asarray(blk.data)), all subsequent operations — unit conversion, tensor reordering, weighting by IVOL/EVOL, and summation — run as compiled, vectorised NumPy operations over the whole block at once (e.g. (data * weight[:, None]).sum(axis=0)) instead of as a Python for-loop with per-element multiplication and addition.
- Locality with the weighting field: because the corresponding weight field (element volume IVOL / EVOL) can also be pulled as bulk data blocks aligned with the same elements, the weighted sum – average operation itself becomes a single array multiply-and-reduce rather than a per-value dictionary/lookup-and-accumulate.

8.3 How it is used in the developed extractor

The kernel module developed for this project (data_extractor.py) applies this strategy uniformly for every averaging path it implements:

- For integration-point and centroidal variables (stress, strain, PEEQ, temperature, …): fo.getSubset(region=phase_region, position=...) returns a FieldOutput restricted to one phase;

its .bulkDataBlocks and the matching IVOL/EVOL blocks are both converted with np.asarray(...) and combined as Σ(data × volume) / Σ(volume) per block, accumulated across blocks.

- For whole-element variables (ELSE, ELEN, NFORC, …, which are already per-element totals rather than densities): the block data is summed directly (not volume-weighted) and divided by the summed element volume to give a density-style average, matching how the original hand-written script treated ELSE.
- For nodal / element-nodal variables (displacements U, …): bulk .data is read together with the parallel .nodeLabels array and mapped to phases through a pre-built (instance, node label) → phase lookup set, still processed as whole NumPy arrays per block rather than value-by-value.
- Tensor components (stress- and strain-like 6-component variables) are reordered from Abaqus's storage order to Mandel–Kelvin order and scaled in a single vectorised operation (to_mandel_kelvin), instead of re-ordering component-by-component inside the accumulation loop.

The net effect is that adding a new requested variable only changes which of these five generic strategies is dispatched to (decided automatically from the variable's .position and .type, see Section 10.2); it never re-introduces a per-value Python loop, so the speed benefit of the bulk-data-block approach is preserved for every variable the tool supports.

9. Field Quantities Computed and the Governing Equations

The plug-in is deliberately generic (Section 10.2), but the quantities most relevant to RVE homogenisation — and used as the default set in the GUI (S, E) — are summarised below, together with the averaging equation used for each.

9.1 Volume-averaged stress and strain

For a phase p occupying sub-volume V_p of the RVE, the per-phase volume-weighted average of a tensor field σ (e.g. Cauchy stress S, or strain E) evaluated at integration points is:

$$\langle \sigma_{ij} \rangle_p = (1/V_p)\, \Sigma_{g \in p}\, \sigma_{ij}(g)\, V_g \quad (3)$$

where the sum runs over all integration points g belonging to phase p, and V_g (the field output IVOL) is the volume associated with that integration point. The overall, whole-RVE average is obtained by summing the volume-weighted numerator and the volume denominator across all phases before dividing — not by averaging the already-averaged phase values — so that phases are correctly weighted by their true volume:

$$\langle \sigma_{ij} \rangle_{RVE} = \Sigma_p\, \Sigma_{g \in p}\, \sigma_{ij}(g)\, V_g \,/\, \Sigma_p\, \Sigma_{g \in p}\, V_g \quad (4)$$

The same expression, with the corresponding EVOL weight, is used for centroidal quantities such as heat flux (HFL) or section forces.

9.2 Whole-element (already-integrated) quantities

Quantities such as element strain energy (ELSE), kinetic energy (ELEN) or plastic dissipation (ELPD) are stored by Abaqus as one already-integrated total per element, not as a density per unit volume. These are

therefore summed rather than volume-weighted, and only the resulting sum is normalised by the total volume to give a density-style average:

$$\langle w \rangle = \Sigma_e \, ELSE_e \,/\, \Sigma_e \, V_e \quad (5)$$

9.3 Von Mises equivalent stress and principal values

From the Mandel–Kelvin ordered, volume-averaged stress tensor, the plug-in's helper routine assembles the 3×3 symmetric stress matrix and extracts its eigenvalues (principal stresses $\sigma_1 \geq \sigma_2 \geq \sigma_3$) and the von Mises equivalent stress:

$$\sigma_{vM} = \sqrt{(\tfrac{1}{2}[(\sigma_1-\sigma_2)^2+(\sigma_2-\sigma_3)^2+(\sigma_3-\sigma_1)^2])} \quad (6)$$

9.4 Tensor storage convention — Mandel–Kelvin ordering

Abaqus stores symmetric 6-component tensors in the order [11, 22, 33, 12, 13, 23]. The plug-in re-orders every such tensor into Mandel–Kelvin order [11, 22, 33, 23, 13, 12] and rescales the shear components so that ordinary tensor-norm and eigenvalue operations can be applied directly, using √2 scaling for stress-like tensors and 1/√2 scaling for engineering-strain-like tensors:

$$\tilde{\sigma}_i = \sqrt{2} \cdot \sigma_i \quad (7a) \quad \text{— shear components, stress-like}$$

$$\tilde{\varepsilon}_i = \varepsilon_i / \sqrt{2} \quad (7b) \quad \text{— shear components, strain-like}$$

9.5 Volume fraction of each phase

$$\varphi_p = V_p / V_{RVE} \quad (8)$$

9.6 Surface-averaged quantities

For contact/surface variables (e.g. CPRESS, CSTRESS, CDISP), which are not tied to a material phase, the plug-in performs an area-weighted average over the surface region using the contact-area field CNAREA as the weight, analogous to Equation (3) with area replacing volume:

$$\langle f \rangle_{surf} = \Sigma_g \, f(g) \, A_g \,/\, \Sigma_g \, A_g \quad (9)$$

9.7 History and whole-model / modal quantities

Purely global scalars that Abaqus does not decompose spatially (eigen-frequencies EIGFREQ, load-proportionality factor LPF, energy totals such as ALLIE/ALLSE/ALLKE, structural-optimisation quantities such as SOF) are read directly from the step's history output rather than field output, since these have no meaningful per-phase decomposition.

9.8 Effective (homogenised) stiffness — identified future extension

The third tab of the GUI ("Effective properties") is reserved for computing the RVE's homogenised stiffness tensor from the averaged stress and strain of Section 9.1, using the standard definition:

$$\langle \sigma_{ij} \rangle = C_{ijkl}^{eff} \, \langle \varepsilon_{kl} \rangle \quad (10)$$

obtained by solving the RVE under a set of independent unit load cases (e.g. six independent strain states for 3-D elasticity) and assembling one column of C^eff from the averaged stress response to each case, subject to the Hill–Mandel condition of Equation (2). This tab is currently a placeholder in the implementation and is identified in Section 13 as the natural next feature to build on top of the already-generalised averaging engine.

10. Plug-in Architecture and Implementation

10.1 Overview

The tool is delivered as a standard two-file Abaqus/CAE GUI plug-in plus a kernel-side computation module:

File	Role
my_gui_plugin.py	Registers the plug-in with the Abaqus GUI Toolset (registerGuiMenuButton) so it appears under Plug-ins in Abaqus/CAE, and points it at the dialog class defined in my_gui.py.
my_gui.py	Defines MyGuiForm (an AFXForm) and MyGuiDialog (an AFXDataDialog) — the actual three-tab GUI window, its widgets, and the event handlers that build and send kernel commands.
data_extractor.py	Kernel-side module, imported and invoked from the GUI's event handlers via sendCommand(); contains the variable classifier, the five generic averaging strategies (Section 8.3), and the ODB pre-processing / summary routine.
microsolidics-logo.png	Company logo, loaded and scaled at run time into the header banner of the GUI dialog.

10.2 GUI dialog (my_gui.py)

MyGuiDialog subclasses AFXDataDialog and is organised as a header banner (company logo, or a text fallback if the image file is not found beside the script) followed by a three-tab FXTabBook:

- Tab 1 – “Field average quantities”: the main working tab, containing four grouped sections — (a) ODB selection, with a text field and a Browse… button that opens a native file dialog restricted to *.odb files; (b) step name and output .txt file name entry; (c) a Pre-process group with a button that inspects the chosen ODB and displays, in a read-only text box, the available field-output variable names, the detected phases (instance / section / material) and the number of steps and frames; and (d) an Extraction & Computation group where the user types a comma-separated list of field variables and presses Compute.
- Tab 2 – “Second moments”: placeholder tab reserved for future extension (e.g. spatial second-moment / correlation-function based microstructure descriptors).
- Tab 3 – “Effective properties”: placeholder tab reserved for the effective-stiffness computation described in Section 9.8.

Three button callbacks are mapped through FXMAPFUNC: onBrowseOdb (native file browser via Tkinter, since AFX's own file dialogs are not always convenient for this workflow), onPreProcess, and onCompute. Both onPreProcess and onCompute do not run heavy Abaqus/odbAccess code directly inside the GUI process; instead they build a small Python command string and dispatch it into the Abaqus kernel process with sendCommand(...), then read back a result written to a temporary text file — the standard Abaqus GUI/kernel separation pattern, which keeps the GUI responsive while the (potentially large) ODB is processed in the kernel.

10.3 Kernel extraction module (data_extractor.py)

The kernel module is deliberately written as a generalized dispatcher rather than one hand-written code path per variable. For a requested variable name, classify_variable() inspects frame.fieldOutputs[varname].locations[i].position and .type at run time to determine where the data is stored (integration point, centroid, whole-element, element-face, nodal/element-nodal, or — if the name is not a field output at all — a known whole-model history quantity), and returns a small VarInfo record. Based on this classification the appropriate one of the five averaging strategies from Section 8.3 is invoked, and results are accumulated per phase and per frame, then written to a structured output text file containing: the requested variables and any warnings for variables not found in the ODB; per-phase results with volume fractions and (for tensors) values labelled in Mandel–Kelvin component order; and, for history/surface variables, their own dedicated sections. The same module also implements get_odb_preprocess_info(), used by the GUI's Pre-process button, which opens the ODB read-only and reports the available field-output names, detected phases and step/frame counts without running any averaging.

10.4 Company branding

A short, defensive header-banner block was added at the top of the dialog constructor: it looks for microsolidics-logo.png in the same directory as the script, loads and proportionally rescales it (afxCreatePNGIcon, capped to a maximum width so the dialog is not dominated by the logo) and displays it in an FXLabel; if the file is missing, it falls back to a plain text label reading “MICROSOLIDICS” so the dialog still opens cleanly on another machine where the logo file was not copied alongside the scripts.

11. Typical Workflow

1. Generate the RVE microstructure (Section 4) and apply the appropriate boundary conditions (Section 5); solve the job in Abaqus/Standard or Abaqus/Explicit to obtain an .odb.
2. Open Abaqus/CAE, launch the plug-in from the Plug-ins menu (Data Extractor GUI).
3. On Tab 1, click Browse… and select the .odb file.
4. Click Pre-process to list the available field-output variables and the detected material phases before deciding what to average.
5. Enter the analysis Step Name, the desired output .txt file name, and the comma-separated list of field variables to average (e.g. S,E,PEEQ,SENER).

6. Click Compute; the kernel-side generalized extractor classifies each variable, applies the corresponding bulk-data-block averaging strategy per phase and per frame, and writes the structured results file.
7. Use the written output text file (overall averages, per-phase averages with volume fractions, and any missing-variable warnings) directly in further homogenisation, calibration or reporting work.

12. Discussion

The central engineering outcome of this project is architectural rather than a single numerical result: by re-routing every averaging path through bulkDataBlocks and NumPy instead of a per-value Python loop, the same post-processing task that previously required one hand-written code block per variable name now runs, for any supported variable, through one of five generic and equally fast strategies chosen automatically at run time. This directly addresses the requirement identified in Sections 3 and 7 — that batch, high-resolution RVE studies are only practical if post-processing does not become the new bottleneck once the solve itself has been made efficient. Because the bulk-data-block calls scale with the number of blocks (essentially the number of instances / element types) rather than with the number of integration points or nodes, the relative speed advantage over the conventional loop widens as the RVE mesh is refined — precisely the regime in which accurate homogenisation (per Kanit et al. [2]) demands the largest models.

A secondary, and practically important, outcome is usability: because the classification and averaging logic is generic, a user of the GUI can request a new field variable (say, a heat-flux or contact-pressure quantity) without any code changes, simply by typing its name in the Variables field — the tool determines automatically how that variable is stored and how it should be weighted and averaged.

13. Conclusion and Future Scope

13.1 Conclusion

A fully functional Abaqus/CAE plug-in was developed that automates the extraction and volume-averaging of arbitrary field-output quantities from RVE simulations, replacing the conventional, slow, per-value Python loop with a generalized, bulk-data-block-based, vectorised NumPy implementation. The plug-in couples a three-tab AFX GUI dialog (ODB browsing, ODB pre-processing/inspection, and variable-driven computation) with a kernel-side classification-and-dispatch engine covering integration-point, centroidal, whole-element, nodal and surface field quantities, as well as history/modal outputs, and reports results per material phase with volume fractions in a structured text file. The bulk-data-block strategy was adopted specifically to meet the requirement of making repeated, high-resolution RVE post-processing fast enough to be used routinely in multiscale material-design studies.

13.2 Future Scope

- Implement Tab 3 (“Effective properties”) to solve the standard set of unit load cases and assemble the homogenised stiffness tensor C^eff of Equation (10) directly from the already-generalised averaging engine.

- Implement Tab 2 (“Second moments”) to compute spatial second-moment / two-point correlation descriptors of the microstructure, useful for RVE-size and representativity studies in the spirit of Kanit et al. [2].
- Extend the bulk-data-block strategy to a batch mode that loops over many .odb files (e.g. an RVE-size convergence study or a design-of-experiments sweep) in a single kernel session, further amortising start-up overhead.
- Add quantitative benchmarking (wall-clock time vs. mesh size, conventional loop vs. bulk-data-block) to the tool's own output, so that the speed-up documented qualitatively in this report is logged automatically for every run.
- Package the pre-processing (microstructure generation) and boundary-condition tools (Sections 4–5) into the same plug-in so that generation, solving and post-processing form one continuous, GUI-driven pipeline.

References

[1] Hill, R. (1963). “Elastic properties of reinforced solids: some theoretical principles.” Journal of the Mechanics and Physics of Solids, 11(5), 357–372.

[2] Kanit, T., Forest, S., Galliet, I., Mounoury, V., Jeulin, D. (2003). “Determination of the size of the representative volume element for random composites: statistical and numerical approach.” International Journal of Solids and Structures, 40(13–14), 3647–3679.

[3] Suquet, P. M. (1987). “Elements of homogenization for inelastic solid mechanics.” In: Homogenization Techniques for Composite Media (E. Sanchez-Palencia and A. Zaoui, eds.), Lecture Notes in Physics, Springer, Berlin, pp. 193–198.

[4] Nemat-Nasser, S., Hori, M. (1999). Micromechanics: Overall Properties of Heterogeneous Materials, 2nd ed. Elsevier, Amsterdam.

[5] Zohdi, T. I., Wriggers, P. (2008). An Introduction to Computational Micromechanics. Lecture Notes in Applied and Computational Mechanics, Springer, Berlin.

[6] Segurado, J., Llorca, J. (2002). “A numerical approximation to the elastic properties of sphere-reinforced composites.” Journal of the Mechanics and Physics of Solids, 50(10), 2107–2121.

[7] Michel, J. C., Moulinec, H., Suquet, P. (1999). “Effective properties of composite materials with periodic microstructure: a computational approach.” Computer Methods in Applied Mechanics and Engineering, 172(1–4), 109–143.

[8] Xia, Z., Zhou, C., Yong, Q., Wang, X. (2006). “On selection of repeated unit cell model and application of unified periodic boundary conditions in micro-mechanical analysis of composites.” International Journal of Solids and Structures, 43(2), 266–278.

[9] Tian, W., Qi, L., Chao, X., Liang, J., Fu, M. (2019). “Periodic boundary condition and its numerical implementation algorithm for the evaluation of effective mechanical properties of the composites with complicated micro-structures.” Composites Part B: Engineering, 162, 1–10.

[10] Dassault Systèmes Simulia Corp. Abaqus Scripting User's Guide and Abaqus Scripting Reference Guide. Abaqus Documentation (odbAccess module: FieldOutput.bulkDataBlocks).

[11] Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). “Array programming with NumPy.” Nature, 585, 357–362.

[12] Feyel, F., Chaboche, J.-L. (2000). "FE² multiscale approach for modelling the elastoviscoplastic behaviour of long fibre SiC/Ti composite materials." Computer Methods in Applied Mechanics and Engineering, 183(3–4), 309–330.

[13] Geers, M. G. D., Kouznetsova, V. G., Brekelmans, W. A. M. (2010). "Multi-scale computational homogenization: Trends and challenges." Journal of Computational and Applied Mathematics, 234(7), 2175–2182.